

Módulo 3. Primeros pasos con Google Colab y TensorFlow

Ejemplo 1. Escribir un programa que pregunte el nombre del usuario en la consola y después de que el usuario lo introduzca muestre por pantalla la cadena ¡Hola <nombre>!, donde <nombre> es el nombre que el usuario haya introducido.

Código:

```
nombre = input("Introduce tu nombre: ")
print("¡Hola " + nombre + "!")
```

Ejemplo 2. Escribir un programa que lea un entero positivo, n , introducido por el usuario y después muestre en pantalla la suma de todos los enteros desde 1 hasta n . La suma de los n primeros enteros positivos puede ser calculada de la siguiente forma:

$$suma = \frac{n(n + 1)}{2}$$

Código:

```
n = int(input("Introduce un número entero: "))
suma = n * (n + 1) / 2
print("La suma de los primeros números enteros desde 1 hasta " + str(n) +
      " es " + str(suma))
```

Explicación detallada:

```
n = int(input("Introduce un número entero: "))
```

- `input("Introduce un número entero: ")`: muestra en pantalla el mensaje "Introduce un número entero: " y espera que el usuario escriba algo y presione Enter. Lo que el usuario escriba se recibe como una **cadena de texto (string)**.
- `int(...)`: convierte esa cadena de texto ingresada por el usuario en un número entero.
- `n =`: asigna ese número entero a la variable llamada `n`.

```
suma = n * (n + 1) / 2
```

Esta línea calcula la suma de los primeros n números enteros positivos, usando la fórmula matemática conocida. Se multiplica n por $n + 1$, luego se divide entre 2. El resultado se guarda en la variable `suma`.

```
print("La suma de los primeros números enteros desde 1 hasta " + str(n) +  
" es " + str(suma))
```

- `print(...)`: muestra en pantalla el texto que va dentro del paréntesis.
- Se concatenan varias cadenas de texto (`str(...)`) y mensajes:
 - "La suma de los primeros números enteros desde 1 hasta " es un texto fijo.
 - `str(n)` convierte el número entero `n` en texto para poder concatenarlo.
 - " es " es otro texto fijo.
 - `str(suma)` convierte el resultado `suma` en texto.
- El resultado es una frase completa que indica el valor calculado, por ejemplo:
"La suma de los primeros números enteros desde 1 hasta 10 es 55.0"

Ejemplo 3. Escribir un programa que pida al usuario su peso (en kg) y estatura (en metros), calcule el índice de masa corporal y lo almacene en una variable, y muestre por pantalla la frase Tu índice de masa corporal es <imc> donde <imc> es el índice de masa corporal calculado redondeado con dos decimales.

Código:

```
peso = input("¿Cuál es tu peso en kg? ")  
estatura = input("¿Cuál es tu estatura en metros?")  
imc = round(float(peso)/float(estatura)**2,2)  
print("Tu índice de masa corporal es " + str(imc))
```

Explicación detallada:

```
peso = input("¿Cuál es tu peso en kg? ")
```

- `input(...)` muestra en pantalla el mensaje "¿Cuál es tu peso en kg? " y espera que el usuario escriba su peso.
- Lo que el usuario escriba se guarda como una **cadena de texto (string)** en la variable `peso`.

```
estatura = input("¿Cuál es tu estatura en metros?")
```

- Es similar a la línea anterior, muestra el mensaje "¿Cuál es tu estatura en metros?" y espera que el usuario ingrese su estatura.
- El valor ingresado se guarda como cadena de texto en la variable `estatura`.

```
imc = round(float(peso)/float(estatura)**2, 2)
```

- Primero, convierte `peso` (cadena) a un número decimal (float) con `float(peso)`.
- Convierte `estatura` (cadena) a float con `float(estatura)`.
- Calcula el cuadrado de la estatura: `float(estatura)**2`.
- Divide el peso entre el cuadrado de la estatura para calcular el Índice de Masa Corporal (IMC).
- La función `round(..., 2)` redondea el resultado a **2 decimales**.
- El resultado final se asigna a la variable `imc`.

```
print("Tu índice de masa corporal es " + str(imc))
```

- Convierte `imc` a cadena con `str(imc)` para poder concatenarla con texto.
- Muestra en pantalla el mensaje completo, por ejemplo:
"Tu índice de masa corporal es 23.45"

Ejemplo 4. Creación de tensores simples (tensores escalares)

Código:

```
import tensorflow as tf

# Crear tensores escalares
a = tf.constant(5)
b = tf.constant(3)

print("Tensor a:", a)
print("Tensor b:", b)
```

Ejemplo 5. Operaciones básicas entre tensores escalares

Código:

```
import tensorflow as tf

# Crear tensores escalares
a = tf.constant(5)
b = tf.constant(3)

# Suma, resta, multiplicación, división
suma = tf.add(a, b)
resta = tf.subtract(a, b)
producto = tf.multiply(a, b)
division = tf.divide(a, b)

print("Tensor a:", a)
print("Tensor b:", b)
print("Suma:", suma.numpy())
print("Resta:", resta.numpy())
print("Multiplicación:", producto.numpy())
print("División:", division.numpy())
```

Explicación detallada:

```
import tensorflow as tf
```

- Importa la biblioteca **TensorFlow** y la referencia como `tf`.
- TensorFlow es una biblioteca para cálculo numérico y computación con tensores.

```
# Crear tensores escalares
a = tf.constant(5)
b = tf.constant(3)
```

- `tf.constant(5)` crea un tensor escalar con valor 5. Un tensor escalar es simplemente un número sin dimensión.
- Se asigna ese tensor a la variable `a`.
- De forma similar, `b` es un tensor escalar con valor 3.

```
# Suma, resta, multiplicación, división
suma = tf.add(a, b)
resta = tf.subtract(a, b)
producto = tf.multiply(a, b)
division = tf.divide(a, b)
```

Aquí se realizan operaciones matemáticas básicas entre tensores `a` y `b` usando funciones de TensorFlow y cada resultado se guarda en sus respectivas variables.

- o `tf.add(a, b)`: suma los tensores y devuelve un tensor con el resultado.
- o `tf.subtract(a, b)`: resta `b` de `a`.
- o `tf.multiply(a, b)`: multiplica `a` por `b`.
- o `tf.divide(a, b)`: divide `a` entre `b`.

```
print("Tensor a:", a)
print("Tensor b:", b)
```

En esta línea se muestran los tensores `a` y `b`.

```
print("Suma:", suma.numpy())
print("Resta:", resta.numpy())
print("Multiplicación:", producto.numpy())
print("División:", division.numpy())
```

En esta parte, se muestran los valores numéricos de los tensores resultado. Se usa `.numpy()`, que convierte el tensor en un valor NumPy (o un número Python estándar en este caso).

Ejemplo 6. Creación de tensores multidimensionales (matrices)

Código:

```
import tensorflow as tf
# Crear una matriz 2x2
matriz = tf.constant([[1, 2], [3, 4]])
print("Matriz:\n", matriz)
```

Ejemplo 7. Calcular el promedio de calificaciones

Código:

```
import tensorflow as tf
# Lista de calificaciones
calificaciones = tf.constant([70, 85, 90, 60, 100], dtype=tf.float32)
promedio = tf.reduce_mean(calificaciones)

print("Calificaciones:", calificaciones.numpy())
print("Promedio:", promedio.numpy())
```

Ejemplo 8. Visualización del gráfico de una función matemática

Código:

```
import numpy as np
import matplotlib.pyplot as plt

# Crear valores de entrada
x_vals = np.linspace(-5, 5, 100)
y_vals = tf.square(x_vals) #  $y = x^2$ 

plt.plot(x_vals, y_vals, label='y = x2')
plt.title('Visualización de una función cuadrática')
plt.xlabel('x')
plt.ylabel('y')
plt.grid(True)
plt.legend()
plt.show()
```

Explicación detallada:

```
import numpy as np
import matplotlib.pyplot as plt
```

- Importa la biblioteca **NumPy** como **np**, que sirve para manipulación numérica y vectores/matrices.
- Importa **Matplotlib**, módulo **pyplot** como **plt**, para graficar datos.

```
# Crear valores de entrada
x_vals = np.linspace(-5, 5, 100)
```

En esta línea, se usa `np.linspace(-5, 5, 100)` para crear un arreglo NumPy con **100 números equidistantes entre -5 y 5**. Este arreglo representa los valores de la variable independiente x sobre los que se evaluará la función.

```
y_vals = tf.square(x_vals) #  $y = x^2$ 
```

En esta línea, se calcula el cuadrado de cada valor de `x_vals` usando la función `tf.square` de TensorFlow. Esto genera un tensor con los valores de $y = x^2$ para cada x de `x_vals`.

```
plt.plot(x_vals, y_vals, label='y = x2')
```

En esta parte, se grafica la curva usando `x_vals` como eje X y `y_vals` como eje Y. La etiqueta ' $y = x^2$ ' se usará para la leyenda.

```
plt.title('Visualización de una función cuadrática')
plt.xlabel('x')
plt.ylabel('y')
plt.grid(True)
plt.legend()
```

En esta parte, se definen las características del gráfico de la función: título del gráfico, etiqueta de los ejes X e Y. Además, se activa la grilla para facilitar la lectura y se muestra la leyenda con la etiqueta que se definió en `plt.plot`.

```
plt.show()
```

Esta línea muestra la ventana o figura con el gráfico generado.

Ejemplo 9. Creación de un tensor con valores del 0 al 9 y elevarlo al cuadrado Código:

```
import tensorflow as tf

x = tf.range(10) # tensor con valores del 0 al 9
y = tf.square(x) # elevar al cuadrado

print("x:", x.numpy())
print("x2:", y.numpy())
```

Ejemplo 10. Cálculo de la media, varianza y desviación estándar Código:

```
import tensorflow as tf
valores = tf.constant([2.0, 4.0, 4.0, 4.0, 5.0, 5.0, 7.0, 9.0])
media = tf.reduce_mean(valores)
varianza = tf.math.reduce_variance(valores)
desviacion = tf.math.reduce_std(valores)
```

```
print("Media:", media.numpy())
print("Varianza:", varianza.numpy())
print("Desviación estándar:", desviacion.numpy())
```

Ejemplo 11. Función definida por el usuario y su gráfico Código:

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

def funcion(x):
    return tf.math.sin(x) * tf.exp(-0.1 * x)

x_vals = tf.linspace(0.0, 10.0, 100)
y_vals = funcion(x_vals)

plt.plot(x_vals, y_vals, label='f(x) = sin(x) * exp(-0.1x)')
plt.title("Función personalizada")
plt.grid(True)
plt.legend()
plt.show()
```

Explicación detallada:

```
import tensorflow as tf
import matplotlib.pyplot as plt
```

- Importa la biblioteca **TensorFlow** como `tf`, para operaciones con tensores y funciones matemáticas.
- Importa **Matplotlib** módulo `pyplot` como `plt`, para crear gráficos.

```
def funcion(x):
    return tf.math.sin(x) * tf.exp(-0.1 * x)
```

En esta parte, se define una función llamada `funcion` que toma como parámetro `x` (un tensor o array). Dentro de la función: calcula el seno de `x` con `tf.math.sin(x)`, calcula $e^{-0.1x}$ con `tf.exp(-0.1 * x)` y multiplica ambos resultados elemento a elemento. Por último, retorna el tensor resultante.

```
x_vals = tf.linspace(0.0, 10.0, 100)
```

En esta línea se usa `tf.linspace` para crear un tensor con **100 valores igualmente espaciados entre 0.0 y 10.0**. Estos valores serán el dominio donde se evaluará la función.

```
y_vals = funcion(x_vals)
```

En esta línea, se evalúa la función `funcion` en cada punto del tensor `x_vals`. El resultado es un tensor `y_vals` con los valores correspondientes de la función $f(x) = \sin(x)e^{-0.1x}$

```
plt.plot(x_vals, y_vals, label='f(x) = sin(x) * exp(-0.1x)')
```

En esta parte, se grafica la curva usando `x_vals` como eje X y `y_vals` como eje Y. Se asigna la etiqueta `'f(x) = sin(x) * exp(-0.1x)'` para la leyenda.

```
plt.title("Función personalizada")
```

Esta línea lo que hace es añadir el título "Función personalizada" al gráfico.

```
plt.grid(True)
```

En esta línea se activa la grilla en el gráfico para facilitar la lectura de valores.

```
plt.legend()
```

En esta línea se muestra la leyenda que incluye la etiqueta definida en `plt.plot`.

```
plt.show()
```

En esta línea se muestra la ventana o figura con el gráfico generado.

Ejemplo 12. Uso de ciclo for en Tensorflow (cálculo de una suma acumulada) Código:

```
import tensorflow as tf
numeros = tf.constant([1, 2, 3, 4, 5])
suma = tf.constant(0)

for n in numeros:
    suma += n

print("Suma total:", suma.numpy())
```

Explicación detallada:

```
import tensorflow as tf
```

Se importa la biblioteca **TensorFlow** y la asigna al alias `tf`.

```
numeros = tf.constant([1, 2, 3, 4, 5])
```

- `tf.constant([...])` crea un **tensor constante** con los valores `[1, 2, 3, 4, 5]`.
- Este tensor es inmutable y representa una **lista de enteros**.

- Se asigna a la variable `numeros`.

```
suma = tf.constant(0)
```

En esta línea, se crea otro tensor constante con el valor inicial 0. Este tensor servirá como acumulador para guardar la suma total.

```
for n in numeros:
    suma += n
```

- Este bucle ~~for~~ recorre cada elemento del tensor `numeros`.
- En cada iteración:
 - `n` toma un valor individual del tensor, por ejemplo: 1, luego 2, etc.
 - `suma += n` **suma el valor actual `n` al acumulador `suma`.**
 - Aunque `suma` se definió como un `tf.constant`, esta operación genera internamente nuevos tensores cada vez que se ejecuta `suma += n` (ya que los tensores constantes no pueden modificarse directamente, pero el operador `+=` crea uno nuevo y lo reasigna).

```
print("Suma total:", suma.numpy())
```

En esta parte, se imprime el resultado final y `.numpy()` convierte el tensor `suma` en un **número Python estándar (int)** para mostrarlo como texto.

Ejemplo 13. Entrenamiento básico de un modelo de regresión lineal

Código:

```
import tensorflow as tf
# Datos de entrada y salida (y = 2x + 1)
x = tf.constant([1, 2, 3, 4, 5], dtype=tf.float32)
y = tf.constant([3, 5, 7, 9, 11], dtype=tf.float32)

# Variables a ajustar
W = tf.Variable(0.0)
b = tf.Variable(0.0)

# Modelo y función de pérdida
def modelo(x): return W * x + b
def perdida(y_real, y_pred): return tf.reduce_mean(tf.square(y_real - y_pred))

# Entrenamiento simple
opt = tf.optimizers.SGD(learning_rate=0.01)

for i in range(300):
    with tf.GradientTape() as tape:
        y_pred = modelo(x)
        loss = perdida(y, y_pred)
    grad = tape.gradient(loss, [W, b])
    opt.apply_gradients(zip(grad, [W, b]))
```

```
print(f"Modelo entrenado:  $y = \{W.numpy():.2f\}x + \{b.numpy():.2f\}$ ")
```

Ejemplo 14. Clasificación básica usando tf.where

Código:

```
import tensorflow as tf
calificaciones = tf.constant([95, 82, 60, 45, 88, 72])
estatus = tf.where(calificaciones >= 70, "Aprobado", "Reprobado")
print("Calificaciones:", calificaciones.numpy())
print("Estatus:", estatus.numpy())
```